

He revisado la reunión completa. El **plan final que realmente queda acordado** evoluciona bastante respecto a la idea inicial: se descarta empezar directamente por microservicios y se adopta una **modernización incremental en dos grandes etapas**, manteniendo Asgard operativo durante el proceso.

Plan final de refactorización de Asgard Clientes Bolivia

1. Crear un entorno específico para la refactorización.

Se necesita un entorno separado del staging habitual para poder trabajar sin interferir con las pruebas y desarrollos actuales. La opción que queda mejor posicionada es crear un **nuevo servidor/staging como copia de producción**, conectado a los servicios existentes que ya están modernizados.

Para la base de datos se plantean dos alternativas: actualizar la BD actual de pruebas con una copia suficientemente reciente de producción, o crear una instancia independiente. El principal condicionante es **Intercambio Documental**, porque necesita una base de datos con un nombre concreto y puede colisionar con las existentes. Warehouse y las APIs actuales pueden reutilizar sus entornos de pruebas porque ya trabajan con versiones modernas de PHP.

2. No congelar main ni detener el desarrollo normal.

Este punto cambia de forma importante el planteamiento inicial.

Asgard debe seguir evolucionando mientras se realiza la modernización. Por tanto, hay que evitar una rama de refactorización que permanezca abierta durante meses y termine provocando un merge masivo.

La estrategia acordada es realizar **cambios pequeños y PR frecuentes contra main**, manteniendo durante esta fase la compatibilidad con el runtime actual.

3. Primera gran fase: migrar el monolito de PHP 5.x a PHP 8.5.

Este pasa a ser el **objetivo prioritario**.

Antes de rediseñar arquitectura o decidir microservicios, se separan dos riesgos:

- riesgo de **runtime y obsolescencia tecnológica**;
- riesgo de **arquitectura y diseño**.

Primero se elimina el riesgo del runtime.

La migración deberá inventariar y resolver, entre otros elementos detectados durante el análisis:

- aproximadamente **226 archivos afectados por mysql_***;
- short tags;
- APIs eliminadas o incompatibles;
- cifrado antiguo;
- incompatibilidades PHP 5.x → PHP 8.x;
- conexiones e integraciones existentes.

La intención es que la aplicación conserve su comportamiento funcional mientras cambia el runtime.

4. Preparar una red de seguridad antes y durante la migración.

La migración se realizará con una combinación de:

- pruebas de caracterización;
- revisión automática de código;
- análisis de compatibilidad PHP;
- pruebas unitarias donde sean viables;
- pruebas de integración;
- pruebas de los flujos funcionales relevantes;
- comprobación de integraciones con Warehouse, Intercambio Documental y APIs;
- entornos Docker cuando resulte útil para probar diferentes runtimes.

Se menciona expresamente que los tests end-to-end existentes parecen escasos o inexistentes, por lo que habrá que construir un **plan de pruebas específico**.

5. Introducir guardarraíles técnicos.

Durante la transición deben implantarse controles automáticos que impidan introducir nuevo código incompatible con PHP moderno.

Por ejemplo:

- nuevas llamadas mysql_*;
- nuevos short tags;

- APIs PHP eliminadas;
- construcciones incompatibles con PHP 8.5.

La idea es que **todo desarrollo nuevo que entre en Asgard durante la migración respete ya las nuevas reglas**, evitando aumentar nuevamente la deuda técnica.

6. Hacer la migración por bloques pequeños.

No se pretende modificar todo Asgard en una única operación.

Se seleccionarán áreas controladas del código y se irá aplicando la estrategia de compatibilidad y modernización progresivamente.

Cada bloque debe:

1. mantener el comportamiento existente;
2. pasar las pruebas;
3. ser compatible con la estrategia de migración;
4. integrarse mediante un PR pequeño;
5. incorporarse a main.

De esta forma se reduce el riesgo de un gigantesco merge al final del proyecto.

7. Realizar primero un piloto.

Antes de desplegar esta estrategia sobre todo Asgard se acuerda:

identificar un área suficientemente pequeña donde probar el mecanismo completo.

El objetivo del piloto es validar:

- compatibilidad PHP;
- metodología de transformación;
- pruebas;
- PR y merge;
- funcionamiento en staging;
- conexiones con el resto del ecosistema.

Warehouse apareció inicialmente como candidato, pero posteriormente se constató que Warehouse ya utiliza una versión moderna de PHP, por lo que **el módulo piloto definitivo todavía debe seleccionarse**.

? Este es uno de los puntos que la reunión deja pendiente.

8. Una vez preparada toda la aplicación, hacer el cambio de runtime.

Después de completar las transformaciones compatibles y validarlas:

entorno de refactorización → staging → producción.

En cada paso deberán ejecutarse nuevamente las pruebas.

El paso a producción debe disponer de **rollback**, conservando una réplica/configuración que permita volver al estado anterior si aparece un problema grave.

Segunda gran fase: modernizar la arquitectura

Una vez Asgard esté funcionando en PHP 8.5, comienza una segunda transformación distinta.

La decisión que aparece al final de la reunión es **no saltar inmediatamente a microservicios**.

El objetivo intermedio sería evolucionar hacia algo parecido a:

Monolito modular + backend desacoplado + frontend moderno.

Aquí aparece una decisión arquitectónica importante propuesta por Luis Loayza:

- separar claramente **backend** y **frontend**;
- backend orientado a APIs;
- frontend independiente;
- aprovechar y completar la **API de Clientes que ya existe**;
- reducir progresivamente PHP mezclado dentro de las vistas.

Como tecnologías candidatas se mencionan:

Backend

- Laravel / PHP moderno;
- API existente de Clientes, ampliándola.

Frontend

- Vue;
- React;
- Angular.

Durante la conversación **Laravel + Vue** aparece como una combinación especialmente razonable, pero no considero que quede formalmente seleccionada como arquitectura definitiva. **?**

9. Modernizar módulo por módulo

Después de estabilizar PHP 8.5, las áreas legacy podrán transformarse progresivamente:

Asgard legacy PHP

↓

PHP 8.5 estable

↓

Separación de lógica

↓

Backend / API

↓

Frontend desacoplado

↓

Monolito modular

Cada nuevo desarrollo debería, en la medida de lo posible, seguir ya esta arquitectura para **no seguir ampliando el patrón legacy de PHP + HTML + acceso a datos mezclados en el mismo archivo.**

10. Microservicios quedan deliberadamente para después

Este es probablemente el cambio estratégico más importante respecto al planteamiento inicial.

No se van a definir ahora los 70 supuestos microservicios detectados por el análisis.

Durante la reunión se reconoce que muchos de esos candidatos probablemente pertenecen al mismo dominio. Por ejemplo:

- componentes específicos de Alicor podrían formar un único dominio;
- elementos de Billing podrían agruparse;
- capacidades de Logistics podrían formar uno o varios dominios.

La decisión es:

primero PHP 8.5 → después modularización y separación frontend/backend → posteriormente decidir qué capacidades merece la pena extraer como microservicios.

Es decir, **microservicios dejan de ser el objetivo inmediato.**

11. Crear y mantener vivo “Asgard Cortex”

En paralelo a todo el proceso se establece otra línea de trabajo que considero estructural.

El repositorio de conocimiento generado durante la auditoría debe mantenerse actualizado e incorporar:

- código;
- esquema y conocimiento de base de datos;
- relaciones explícitas e implícitas;
- reglas de negocio;
- procesos;
- APIs;
- documentación técnica;
- modificaciones realizadas durante la refactorización.

Luis lo denomina en la reunión **“Asgard Cortex”**.

Cada incremento de la refactorización debe producir también documentación que alimente ese repositorio.

El objetivo es que desarrolladores y agentes de IA puedan consultar posteriormente:

qué hace una funcionalidad, dónde está implementada, qué tablas intervienen, qué reglas tiene y qué otros componentes se verían afectados por un cambio.

Esto se considera especialmente relevante para reducir errores cuando se utilice IA para desarrollar sobre Asgard.

La hoja de ruta resultante

La reunión termina, en la práctica, con esta secuencia:

0. Mantener desarrollo normal de Asgard

|



1. Crear entorno independiente de refactorización

|



2. Actualizar / preparar BD de pruebas

|



3. Inventariar incompatibilidades PHP 5.x → PHP 8.5

|



4. Crear pruebas de caracterización + guardarraíles

|



5. Elegir un bloque piloto

|



6. Validar la estrategia completa con el piloto

|



7. Migrar PHP incrementalmente mediante pequeños PR

|



8. Validar continuamente contra main

|



9. Staging completo en PHP 8.5

|



10. Pruebas integrales + rollback preparado

|



11. Producción PHP 8.5

|



12. Separar progresivamente backend / frontend

|



13. Completar y normalizar la API de Clientes

|



14. Evolucionar hacia monolito modular

|



15. Evaluar dominios reales

|



16. Extraer microservicios SOLO cuando aporte valor

En una frase

El plan final es reducir primero el riesgo tecnológico migrando Asgard Clientes Bolivia de PHP 5.x a PHP 8.5 de forma incremental y compatible con el desarrollo cotidiano; después modernizar progresivamente la aplicación separando backend y frontend y modularizando el monolito; y únicamente entonces decidir qué capacidades conviene realmente convertir en microservicios.

Además, Asgard Cortex debe evolucionar en paralelo como repositorio vivo de conocimiento técnico y funcional de todo el sistema.